

LION FIT PRO

Architecture Decision Records (ADR)

1. Introducción

Un Architecture Decisión Record (ADR) es un documento que captura una decisión arquitectónica importante tomada en el contexto de un proyecto de software. Cada ADR describe el contexto del problema, la decisión tomada, la justificación de esa decisión, las alternativas que se consideraron y las consecuencias (positivas y negativas) de la elección.

Este documento compila 8 ADRs del proyecto Lion Fit Pro, cubriendo las decisiones clave de backend, base de datos, autenticación, frontend móvil, comunicación HTTP, despliegue y patrones de seguridad.

Formato de cada ADR:

Campo	Descripción
Contexto	El problema o necesidad que origina la decisión
Decisión	Que se decidió hacer concretamente
Justificación	Por que esta decisión y no otra
Consecuencias	Efectos positivos y negativos de la decisión
Alternativas	Opciones que se evaluaron y descartaron
Referencia de código	Archivos concretos del repositorio donde se aplica la decisión

2. Resumen de ADRs

ID	Título	Fecha	Estado	Tecnología clave
ADR-001	Adopción de NestJS como framework backend	Noviembre 2025	Aceptada	NestJS 11
ADR-002	Uso de Prisma ORM con PostgreSQL	Noviembre 2025	Aceptada	Prisma ORM 6
ADR-003	Autenticación basada en JWT con Passport.js	Noviembre 2025	Aceptada	JWT + Passport
ADR-004	Vue.js 3 + Ionic Framework para la aplicación móvil	Noviembre 2025	Aceptada	Vue.js 3 + Ionic 8
ADR-005	Axios como cliente HTTP con interceptores centralizados	Noviembre 2025	Aceptada	Axios Interceptors

ADR-006	Despliegue en AWS EC2 con servidor Node.js integrado	Noviembre 2025	Aceptada	AWS EC2
ADR-007	Estructura de rutas REST con prefijo global api/v1	Noviembre 2025	Aceptada	REST API v1
ADR-008	Autenticación en el frontend con guards de Vue Router	Noviembre 2025	Aceptada	Vue Router Guards

3. ADRs Detallados

ADR-001 Adopción de NestJS como framework backend

ID	Título	Fecha	Estado	Decisores
ADR-001	Adopción de NestJS como framework backend	Noviembre 2025	Aceptada	Equipo de desarrollo

Contexto

El proyecto requería un backend robusto con soporte nativo para TypeScript, inyección de dependencias, modularidad y facilidad para implementar autenticación JWT y validaciones. Se evaluaron Express.js puro, Fastify y NestJS.

Decisión

Se adopta NestJS 11 como framework principal del backend, aprovechando su arquitectura modular (AppModule, AuthModule, ExerciseModule, WorkoutModule, ExerciseRecordModule), el sistema de guards para protección de rutas y la integración con Passport.js para JWT.

Justificación (Rationale)	Consecuencias	Alternativas consideradas
• Arquitectura modular que mapea directamente al dominio del negocio (un modulo por entidad).	✓ Estructura consistente y predecible entre todos los modulos del backend.	◆ Express.js: Mas ligero pero requiere configuracion manual de todo el ecosistema.
• Inyeccion de dependencias nativa que facilita el testing y la separacion de responsabilidades.	✓ Validacion automatica de DTOs con class-validator y ValidationPipe global.	◆ Fastify: Mayor rendimiento pero menor ecosistema de plugins para autenticacion.
• Integracion lista para usar con class-validator, Passport, JWT y Prisma.	✓ JwtAuthGuard reutilizable con decorador @Public() para rutas abiertas.	◆ Hono: Moderno y rapido pero sin la madurez de NestJS para proyectos empresariales.
• TypeScript de primera clase, consistente con el frontend en Vue.js + TypeScript.	✗ Curva de aprendizaje inicial para desarrolladores sin experiencia con el patron de decoradores.	

Decoradores declarativos (@Controller, @Injectable, @Get, @Post) que reducen el boilerplate.	✗ Mayor overhead que Express puro para APIs muy simples.	
--	--	--

Referencia en el codigo: ProyectoBackend/src/app.module.ts, src/auth/auth.module.ts

ADR-002 Uso de Prisma ORM con PostgreSQL

ID	Titulo	Fecha	Estado	Decisores
ADR-002	Uso de Prisma ORM con PostgreSQL	Noviembre 2025	Aceptada	Equipo de desarrollo

Contexto

El sistema necesita persistir entidades relacionadas: User, Routine, Exercise, Muscle, RoutineExercise, Workout y ExerciseRecord. Se requeria un ORM con migraciones automaticas, tipado fuerte y compatibilidad con NestJS.

Decision

Se utiliza Prisma ORM 6 con PostgreSQL. Cada servicio de dominio extiende PrismaClient directamente (ExerciseService, ExerciseRecordService) e implementa OnModuleInit para gestionar la conexion al inicio del modulo.

Justificacion (Rationale)	Consecuencias	Alternativas consideradas
Schema declarativo en prisma/schema.prisma que sirve como fuente unica de verdad del modelo de datos.	✓ Sin inconsistencias entre el modelo de datos y los tipos TypeScript del backend.	◆ TypeORM: Mayor integracion con NestJS pero decoradores mas verbosos y migraciones menos intuitivas.
Generacion automatica de tipos TypeScript desde el schema, eliminando discrepancias entre BD y codigo.	✓ Migraciones reproducibles: migration.sql versionadas en el repositorio.	◆ Sequelize: Maduro pero sin generacion automatica de tipos TypeScript.
Sistema de migraciones versionadas (4 migraciones identificadas en el proyecto).	✓ deleteMany() para eliminar registros por workoutId en una sola operacion atomica.	◆ Knex (query builder): Mas control pero sin ORM ni tipos automaticos.
Queries type-safe: this.exercise.findMany(), this.exerciseRecord.deleteMany() con autocompletado.	✗ Patron 'extends PrismaClient' por servicio crea multiples instancias de conexion.	
Soporte nativo para relaciones (User->Workout->ExerciseRecord, Exercise->Muscle).	✗ Recomendacion oficial de Prisma es usar un PrismaService singleton injectable.	

Referencia en el codigo: ProyectoBackend/prisma/schema.prisma, src/exercise/exercise.service.ts

ADR-003 Autenticacion basada en JWT con Passport.js

ID	Titulo	Fecha	Estado	Decisores
ADR-003	Autenticacion basada en JWT con Passport.js	Noviembre 2025	Aceptada	Equipo de desarrollo

Contexto

La aplicacion movil necesita autenticar usuarios y proteger rutas. Al ser una SPA movil (Ionic/Capacitor) sin sesiones del servidor, se requeria un mecanismo stateless. Se evaluaron JWT, sesiones con cookies y OAuth2.

Decision

Se implementa autenticacion JWT con @nestjs/jwt y passport-jwt. El token se firma con un secreto configurable via JWT_SECRET (variable de entorno). El frontend almacena el token en localStorage y lo envia en el header Authorization: Bearer. El JwtAuthGuard protege todas las rutas excepto las marcadas con @Public().

Justificacion (Rationale)	Consecuencias	Alternativas consideradas
• JWT es stateless: compatible con despliegue en AWS EC2 sin necesidad de session store.	✓ Arquitectura stateless lista para escalar horizontalmente.	◆ Sesiones con cookies: Estado en servidor, incompatible con arquitectura stateless en AWS.
• El decorador @Public() permite marcar endpoints abiertos (login, register) sin duplicar logica.	✓ JwtAuthGuard reutilizable: una sola configuracion protege todos los modulos.	◆ OAuth2/Keycloak: Mas seguro pero añade complejidad operacional innecesaria para MVP.
• El interceptor de Axios en api.service.ts inyecta automaticamente el token en cada request.	✓ Refresh automatico del header Authorization via interceptor de Axios.	◆ Supabase Auth: SaaS que simplifica la autenticacion pero genera dependencia de terceros.
• bcrypt para hash de contraseñas con salt rounds (seguridad ante ataques de rainbow table).	✗ JWT_SECRET con fallback hardcodeado ('your-secret-key') en jwt.strategy.ts es un riesgo de seguridad en produccion.	
• TokenPayload tipado con { email, sub, name } para evitar accesos arbitrarios al payload.	✗ Sin mecanismo de revocacion de tokens (logout no invalida el JWT en el servidor).	
	✗ Token almacenado en localStorage es vulnerable a ataques XSS.	

Referencia en el codigo: ProyectoBackend/src/auth/strategies/jwt.strategy.ts, src/auth/guards/jwt-auth.guard.ts

ADR-004 Vue.js 3 + Ionic Framework para la aplicacion movil

ID	Titulo	Fecha	Estado	Decisores
----	--------	-------	--------	-----------

ADR-004	Vue.js 3 + Ionic Framework para la aplicacion movil	Noviembre 2025	Aceptada	Equipo de desarrollo
---------	---	----------------	----------	----------------------

Contexto

El proyecto requería una aplicación móvil multiplataforma (Android prioritario) con componentes UI nativos, navegación por tabs y rendimiento aceptable. Se evaluaron React Native, Flutter, Ionic + React e Ionic + Vue.

Decision

Se utiliza Vue.js 3 con Composition API e Ionic 8 como framework de componentes UI móviles, compilado para Android via Capacitor 7. El enrutamiento se maneja con @ionic/vue-router.

Justificacion (Rationale)	Consecuencias	Alternativas consideradas
• Ionic provee componentes nativos (IonPage, IonHeader, IonSelect, IonCard) sin necesidad de implementar UI desde cero.	✓ Un solo codebase para web y Android (multiplataforma).	◆ React Native: Mayor rendimiento nativo pero requiere conocimiento de React.
• Capacitor 7 permite compilar la web app como APK Android con acceso a APIs nativas.	✓ Componentes Ionic manejan estilos nativos según la plataforma automáticamente.	◆ Flutter: Mejor rendimiento pero lenguaje Dart ajeno al equipo.
• Vue 3 Composition API con <script setup> reduce el boilerplate respecto a Options API.	✓ Ecosistema completo: @ionic/vue-router, IonTabs, IonModal incluidos.	◆ Ionic + React: Similar pero el equipo tenía más experiencia con Vue.
• TypeScript integrado en el proyecto (tsconfig.json, vue-tsc en el build).	✗ Rendimiento inferior a React Native o Flutter para animaciones complejas.	
• Vite como bundler para Hot Module Replacement rápido en desarrollo.	✗ El TrainingPage.vue usa datos simulados (registroHoy = null hardcodeado) sin integración real con el backend.	

Referencia en el código: Proyecto-movil-Frontend/src/views/, src/router/index.ts, capacitor.config.ts

ADR-005 Axios como cliente HTTP con interceptores centralizados

ID	Título	Fecha	Estado	Decisores
ADR-005	Axios como cliente HTTP con interceptores centralizados	Noviembre 2025	Aceptada	Equipo de desarrollo

Contexto

La aplicacion movil necesita comunicarse con el backend REST. Se requeria un cliente HTTP que soporte interceptores para inyeccion automatica del token JWT, manejo centralizado de errores 401 y tipado de respuestas.

Decision

Se utiliza Axios con una instancia centralizada en `api.service.ts`. Se configuran dos interceptores: uno de request que adjunta el Bearer token desde `localStorage`, y uno de response que maneja el error 401 redirigiendo al login y registra en consola todos los errores de API.

Justificacion (Rationale)	Consecuencias	Alternativas consideradas
• Un solo punto de configuracion (baseUrl: <code>http://18.217.60.222:3000/api/v1</code>) para todos los servicios.	✓ DRY principle: la logica de autenticacion se escribe una vez en el interceptor.	◆ Fetch API nativa: Sin dependencias externas pero sin interceptores nativos.
• Interceptor de request elimina la necesidad de pasar el token manualmente en cada llamada.	✓ Facil de extender: agregar headers globales o logging en un solo lugar.	◆ ofetch (unjs): Mas moderno pero menor adopcion en el ecosistema Vue/Ionic.
• Manejo centralizado del error 401 (token expirado o invalido) con limpieza de <code>localStorage</code>.	✗ La URL base esta hardcodeda con IP publica (18.217.60.222) en lugar de variable de entorno.	◆ Apollo Client: Aplica solo si se hubiera usado GraphQL en lugar de REST.
• Todos los servicios (<code>exerciseService</code>, <code>userService</code>, <code>workoutService</code>) importan la misma instancia 'api'.	✗ El interceptor de response tiene <code>console.log</code> en respuestas exitosas (ruido en produccion).	
	✗ <code>useRouter()</code> dentro del interceptor de error es un antipatron fuera del contexto de Vue.	

Referencia en el codigo: `Proyecto-movil-Frontend/src/services/api.service.ts`

ADR-006 Despliegue en AWS EC2 con servidor Node.js integrado

ID	Titulo	Fecha	Estado	Decisores
ADR-006	Despliegue en AWS EC2 con servidor Node.js integrado	Noviembre 2025	Aceptada	Equipo de desarrollo

Contexto

El backend necesitaba un servidor accesible desde internet para que la aplicacion Android pueda conectarse. Se evaluaron AWS EC2, Railway, Render y Heroku como plataformas de despliegue.

Decision

El backend NestJS se despliega en una instancia AWS EC2 en la IP 18.217.60.222, puerto 3000. El NestExpressApplication sirve adicionalmente los archivos estaticos del frontend compilado desde el directorio frontend/dist, consolidando frontend y backend en un unico proceso Node.js.

Justificacion (Rationale)	Consecuencias	Alternativas consideradas
• Control total sobre la instancia: configuracion de puertos, variables de entorno y recursos.	✓ Arquitectura simplificada para MVP: un servidor, una IP, un proceso.	◆ Railway/Render: Despliegue automatico desde Git pero menos control y posibles costos.
• Servir el frontend desde NestJS elimina la necesidad de un servidor web separado (Nginx, Apache).	✓ Sin costos adicionales de CDN o servidor web separado.	◆ AWS ECS + RDS: Arquitectura mas robusta pero complejidad excesiva para un MVP.
• Un unico proceso simplifica el despliegue para un proyecto academico/MVP.	✗ IP publica hardcodeada en el frontend (18.217.60.222) no es una buena practica.	◆ Vercel (frontend) + Railway (backend): Separacion de responsabilidades pero configuracion CORS adicional.
• CORS configurado con origen: '*' permite conexiones desde la app Android y el navegador.	✗ CORS con origen '*' es inseguro para produccion real.	
• Puerto configurable via variable de entorno process.env.PORT con fallback a 3000.	✗ Sin HTTPS: la comunicacion entre la app Android y el servidor no esta cifrada.	
	✗ Sin balanceador de carga ni alta disponibilidad.	

Referencia en el codigo: ProyectoBackend/src/main.ts, Proyecto-movil-Frontend/src/services/api.service.ts

ADR-007 Estructura de rutas REST con prefijo global api/v1

ID	Titulo	Fecha	Estado	Decisores
ADR-007	Estructura de rutas REST con prefijo global api/v1	Noviembre 2025	Aceptada	Equipo de desarrollo

Contexto

El backend expone multiples recursos (user, auth, exercise, workout, routine, routine-exercise, exercise-record). Se requeria una convencion de rutas consistente que soporte versionado futuro y sea legible para el frontend.

Decision

Se aplica un prefijo global 'api/v1' via app.setGlobalPrefix('api/v1') en main.ts. Los controladores

usan rutas de recurso en plural o dominio: /auth, /user, /exercise, /workout, /routine-exercise, /exercise-record. Las rutas anidadas usan parametros: /exercise-record/workout/:workoutId.

Justificacion (Rationale)	Consecuencias	Alternativas consideradas
El prefijo v1 permite introducir v2 en el futuro sin romper clientes existentes.	✓ Versionado de API preparado para evolucionar sin romper la app movil.	◆ Versionado por header (Accept: application/vnd.api+json; version=1): Mas REST puro pero complejo.
Rutas RESTful predecibles: GET /exercise (listar), POST /exercise (crear), DELETE /exercise/:id (eliminar).	✓ Rutas autodescriptivas que facilitan el consumo desde el frontend.	◆ Rutas sin version (/api/exercise): Simple pero sin ruta de migracion para versiones futuras.
El frontend Axios tiene una sola baseURL que incluye el prefijo api/v1.	✗ La ruta /workout/by-date-range podria confundirse con un recurso :id en Express si no se define antes.	◆ GraphQL: Un solo endpoint pero cambio de paradigma completo.
Ruta especializada /workout/by-date-range con query params (userId, startDate, endDate) para consultas de historial.	✗ userId se envia como query param en lugar de extraerse del JWT (implicacion de seguridad).	

Referencia en el codigo: ProyectoBackend/src/main.ts, src/workout/workout.controller.ts

ADR-008 Autenticacion en el frontend con guards de Vue Router

ID	Titulo	Fecha	Estado	Decisores
ADR-008	Autenticacion en el frontend con guards de Vue Router	Noviembre 2025	Aceptada	Equipo de desarrollo

Contexto

La aplicacion Vue necesitaba proteger las rutas autenticadas (/tabs/*) sin duplicar logica de autenticacion en cada componente. Se evaluaron guards de navegacion globales, guards por ruta y composables de autenticacion.

Decision

Se implementa la funcion authGuard en router/index.ts que verifica la existencia del token 'authToken' en localStorage. La funcion se aplica como beforeEnter en cada ruta protegida individualmente: training,

daily-log, exercise/:workoutId y weekly-history.

Justificacion (Rationale)	Consecuencias	Alternativas consideradas
• Guard como funcion pura reutilizable sin acoplamiento a ningun componente Vue.	✓ Logica de proteccion centralizada en un unico lugar del router.	◆ Guard global beforeEach: Mas DRY pero requiere lista de rutas publicas explicita.
• beforeEnter por ruta (no global) da control explicito sobre cuales rutas se protegen.	✓ Facil de extender: agregar roles o permisos modifica solo authGuard.	◆ Pinia store de autenticacion: Estado reactivo pero mayor complejidad para MVP.
• Verificacion simple por existencia del token (sin validar expiracion) suficiente para MVP.	✗ El guard solo verifica existencia del token, no su validez o expiracion.	◆ Composable useAuth(): Mas reutilizable pero requiere refactorizacion de los componentes.
• La capa de verificacion real de expiracion ocurre en el backend via JwtAuthGuard.	✗ Si el token expira, el usuario solo se entera cuando el backend retorna 401.	
	✗ 'authToken' como string magico en vez de constante exportada.	

Referencia en el codigo: Proyecto-movil-Frontend/src/router/index.ts